

FedTeddi: Temporal Drift and Divergence Aware Scheduling for Timely Federated Edge Learning

Yuxuan Bai¹, Yuxuan Sun¹, Tan Chen², Wei Chen¹, Sheng Zhou², Zhisheng Niu²

¹School of Electronic and Information Engineering, Beijing Jiaotong University, Beijing 100044, China

²Beijing National Research Center for Information Science and Technology

Department of Electronic Engineering, Tsinghua University, Beijing 100084, China

Email: {24120024, yxsun}@bjtu.edu.cn, chent21@mails.tsinghua.edu.cn, weich@bjtu.edu.cn, {sheng.zhou, niuzhs}@tsinghua.edu.cn

Abstract—Federated edge learning (FEEL) enables collaborative model training across distributed clients over wireless networks without exposing raw data. While most existing studies assume static datasets, in real-world scenarios, clients may continuously collect data with time-varying and non-independent and identically distributed (non-i.i.d.) characteristics. A critical challenge is how to adapt models in a timely yet efficient manner to such evolving data. In this paper, we propose FedTeddi, a temporal-drift-and-divergence-aware scheduling algorithm that facilitates fast convergence of FEEL under dynamic data evolution and communication resource limits. We first quantify the temporal dynamics and non-i.i.d. characteristics of data using temporal drift and collective divergence, respectively, and represent them as the Earth Mover’s Distance (EMD) of class distributions for classification tasks. We then propose a novel optimization objective and develop a joint scheduling and bandwidth allocation algorithm, enabling the FEEL system to learn from new data quickly without forgetting previous knowledge. Experimental results show that our algorithm achieves higher test accuracy and faster convergence compared to benchmark methods, improving the rate of convergence by 58.4% on CIFAR-10 and 49.2% on CIFAR-100 compared to random scheduling.

Index Terms—Federated edge learning, temporal drift, data heterogeneity, client scheduling, continual learning

I. INTRODUCTION

Federated edge learning (FEEL) enables edge clients to collaboratively train machine learning (ML) models on local data, alleviating data privacy concerns while reducing the transmission and storage burdens of traditional centralized learning [1]. Powered by 6G wireless networks, FEEL is a paradigm shift in ML, enabling various applications such as autonomous driving, logistics services [2] and healthcare [3].

However, current FEEL frameworks predominantly operate under static data assumptions, where the data distribution at each client remains unchanged throughout the training process [4]. This assumption often fails in practical scenarios where edge clients *continuously generate or collect new data* with evolving patterns and distributions. For example, in Internet of Things (IoT) applications, clients such as sensors and

wearables continuously collect data that may change over time due to environmental factors and user behaviors. In autonomous driving, vehicles generate continuously evolving data as they encounter different traffic situations and weather conditions. Accordingly, federated continual learning (FCL) has attracted increasing attention, which enables models to be continuously updated across multiple clients to adapt to time-evolving data distributions while maintaining data privacy [5].

The dynamic nature of real-world environments introduces three fundamental challenges to FEEL [6], [7]. First, when new data arrives, models need to be tuned timely. However, during this process, models are prone to catastrophic forgetting, where previously learned knowledge is lost while acquiring new knowledge. To prevent this, a *balance between plasticity and stability* must be maintained, so that the model can effectively integrate new knowledge while retaining knowledge of the past [4], [8]. Second, data distributions may vary widely, i.e., *non-independent and identically distributed (non-i.i.d.)*, across clients, leading to global model drift and convergence rate degradation [9]–[11]. The introduction of new classes may further exacerbate this heterogeneity, as different clients may collect distinct sets of new classes at different times [12], [13]. Third, when deployed on *resource constrained* mobile networks with limited power, bandwidth and computation, FEEL suffers from high training latency and low model accuracy, particularly under data and system heterogeneity [14], [15]. Limited communication bandwidth necessitates selective client participation in each training round, making it crucial to identify which clients should participate to maximize learning efficiency while satisfying resource constraints [16], [17].

To address the challenges above, we propose a temporal-drift-and-divergence-aware scheduling algorithm for federated edge learning (FedTeddi). Our algorithm explicitly incorporates the effects of data distribution shifts and non-i.i.d. characteristics, ensuring timely model updates under data evolution and resource constraints. The main contributions are:

(1) We quantify *temporal drift* and *collective divergence* using gradient norms, and characterize their impacts on FEEL. Here, temporal drift refers to the change of client data distribution over time, while collective divergence captures the discrepancy between the collectively selected clients and the global data distribution. For classification tasks, both factors

This work is supported in part by the National Natural Science Foundation of China (62301024, U2468201), the Young Elite Scientists Sponsorship Program by China Association for Science and Technology (CAST) under grant 2024-2026QNRC001, the Beijing Natural Science Foundation under grant L222044, and the Talent Fund of Beijing Jiaotong University under grant 2023XKRC030.

are further expressed as the Earth Mover's Distance (EMD) of class distributions in closed-form.

(2) We propose a novel optimization objective to strike a balance between temporal drift and collective divergence, enabling the model to learn timely from new data while mitigating the forgetting of previous knowledge. FedTeddi algorithm is then developed, which jointly optimizes client selection and bandwidth allocation under latency constraints.

(3) Experimental results on CIFAR-10 and CIFAR-100 show that FedTeddi outperforms representative benchmarks in both model accuracy and convergence rate. For example, on CIFAR-10, the convergence rate is increased by 38.5% over pure drift scheduling and 58.4% over random scheduling.

II. RELATED WORK

A. Continual Learning and Its Federated Implementations

Time-evolving data and catastrophic forgetting in continual learning (CL) have been addressed through three paradigms: replay-based methods, where past data is retained for rehearsal or replay efficiency is optimized (e.g., generative replay with meta-learning [18], and efficient replay in federated incremental learning [19]); regularization-based methods, which constrain parameter updates to preserve knowledge without explicit replay (e.g., adaptive importance weighting [20], and rehearsal-free federated learning with synergistic synaptic intelligence [21]); and architecture-based methods, which dynamically expand or allocate sub-networks (e.g., mask-based continual adaptation [22], iterative pruning and retraining [8]).

While high performance has been achieved in centralized settings, additional challenges have been introduced in FL, including asynchronous and correlated concept drift, communication bottlenecks, and heterogeneous clients [10], [11]. To address these challenges, local drift detection is employed in federated learning with concept drift detection (FedConD), along with drift adaptation through regularization parameter adjustment and prudent server-side selection of local updates [23]. Multi-model strategies are leveraged in federated drift adaptation (FedDrift) to tackle distributed drift [24]. Moreover, the Flash method [25] has been proposed to accelerate adaptation to synchronous drift by dynamically adjusting the learning rate. Despite such progress, limitations remain in overcoming communication and resource constraints in edge systems, where clients cannot be scheduled arbitrarily.

B. Federated Edge Learning under Data Heterogeneity and Resource Constraints

In FEEL, significant efforts have been made to address the dual challenges of non-i.i.d. data and resource constraints. Various client scheduling have been designed to mitigate model divergence, accelerate convergence, and adapt to limited bandwidth, energy, and computational capacity [26]. For instance, a heterogeneity-aware sampling mechanism is introduced in federated class-balanced sampling (FedCBS), which alleviates class imbalance and improves accuracy [27]. In vehicular networks, mobility is exploited to accelerate convergence in

hierarchical FL systems [28]. Communication and resource-efficient strategies have been proposed, such as dynamic scheduling for FEEL under latency [16] or energy constraints [29]. Federated collective gradient divergence (FedCGD) optimizes client scheduling by minimizing collective gradient divergences under wireless resource limitations [30]. Furthermore, fairness-aware scheduling mechanisms have been proposed to balance participation among heterogeneous clients [31], while reinforcement learning-based methods enable dynamic client selection under resource constraints [32]. However, most existing approaches still fail to adequately account for dynamic environments where data distributions evolve in real time. In such settings, local datasets at clients arrive continuously and change over time, necessitating the design of adaptive scheduling strategies that can maintain efficiency under strict communication and computation constraints.

III. SYSTEM MODEL

A. Federated Edge Learning with Heterogeneous Data Arrival

Consider a FEEL system with N clients $\mathcal{N} = \{1, 2, \dots, N\}$ and a server, aiming to learn classification tasks timely. As shown in Fig.1, the system operates in a streaming environment, where clients may receive new data at the beginning of each frame as the environment evolves. The training process evolves over L frames, where each frame characterizes a distinct stage of data evolution, and new data are collected and used for training at the beginning of each frame.

Specifically, at the start of frame l , the local dataset of client $n \in \mathcal{N}$ is constructed as $\mathcal{D}_{n,l} = \mathcal{D}_{n,l}^{\text{new}} \cup \mathcal{D}_{n,l-1}^{\text{replay}}$, where $\mathcal{D}_{n,l}^{\text{new}}$ denotes the samples newly collected and labeled upon frame l , and $\mathcal{D}_{n,l-1}^{\text{replay}}$ denotes the representative samples preserved from frame $l-1$ under storage limits. The representative samples can be selected using state-of-the-art strategies [19], [33], [34], which is orthogonal to our work.

In general, the newly collected data in each frame is non-i.i.d. across clients. Data distribution of client n in frame l is represented by $\mathbf{p}_{n,l} = [p_{n,l}^{(1)}, \dots, p_{n,l}^{(C_l)}]$, where $p_{n,l}^{(c)}$ denotes the proportion of class- c samples at client n in frame l . Here, C_l represents the cumulative number of distinct classes that have appeared across all clients up to frame l . The local dataset $\mathcal{D}_{n,l}$ contains $D_{n,l}$ samples and the global dataset in frame l is $\mathcal{D}_l = \bigcup_{n \in \mathcal{N}} \mathcal{D}_{n,l}$ with totally $D_l = \sum_{n=1}^N D_{n,l}$ samples.

Let $f(\mathbf{w}; \mathbf{x}, y)$ be the class-specific loss function associated with model vector $\mathbf{w} \in \mathbb{R}^d$ and data sample $(\mathbf{x}, y)$, where $\mathbf{x}$ is the input feature vector and $y \in \{1, 2, \dots, C_l\}$ is the true class label for C_l -class classification. The local loss of client n in frame l is defined as

$$F_{n,l}(\mathbf{w}, \mathcal{D}_{n,l}) = \sum_{c=1}^{C_l} p_{n,l}^{(c)} \mathbb{E}_{\mathbf{x}|y=c} [f(\mathbf{w}; \mathbf{x}, y)], \quad (1)$$

where $\mathbb{E}_{\mathbf{x}|y=c} [f(\mathbf{w}; \mathbf{x}, y)]$ is taken over local dataset with class label c . Let $\alpha_{n,l} = \frac{D_{n,l}}{\sum_{n \in \mathcal{N}} D_{n,l}}$, and the global loss function is

$$F_l(\mathbf{w}, \mathcal{D}_l) = \sum_{n=1}^N \alpha_{n,l} F_{n,l}(\mathbf{w}, \mathcal{D}_{n,l}). \quad (2)$$

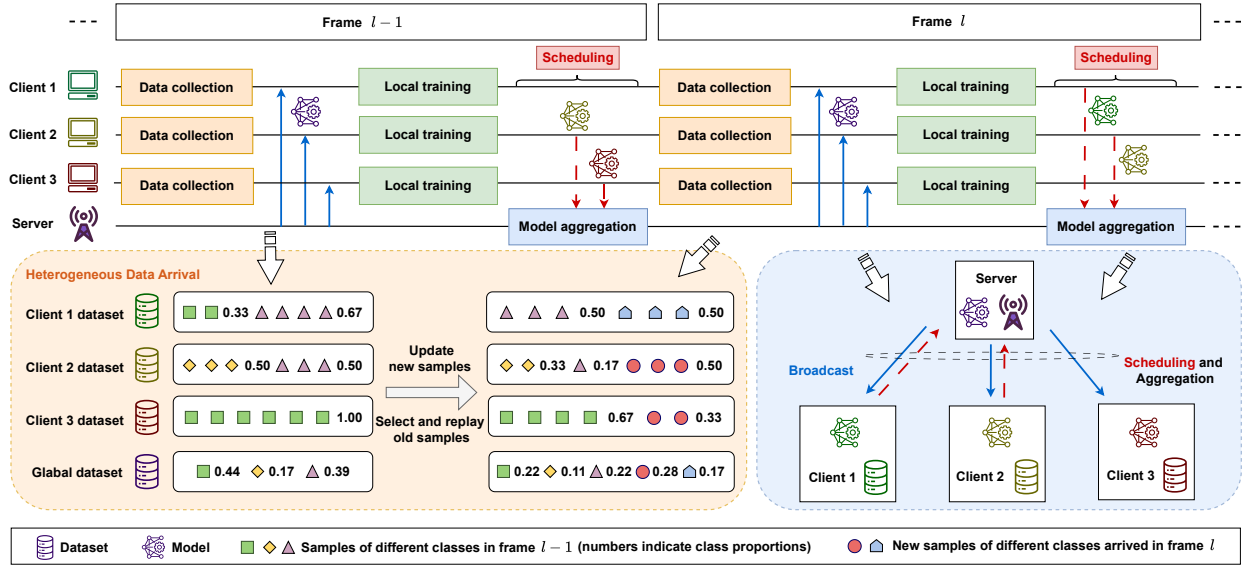


Fig. 1. Schematic illustration of the FEEL system with heterogeneous data arrival across frames.

Frame l consists of K_l rounds of training. In each round k , client n computes τ iterations of the local gradient $G_{n,l,k}^{(t)} = \nabla F_{n,l}(\mathbf{w}_{n,l,k}^{(t)}, \zeta_{n,l,k}^{(t)})$ by running the stochastic gradient descent (SGD) algorithm on a local mini-batch $\zeta_{n,l,k}^{(t)} \subseteq \mathcal{D}_{n,l}$:

$$\mathbf{w}_{n,l,k}^{(t+1)} = \mathbf{w}_{n,l,k}^{(t)} - \eta G_{n,l,k}^{(t)}, t = 0, 1, \dots, \tau - 1, \quad (3)$$

where $\mathbf{w}_{n,l,k}^{(0)} = \mathbf{w}_{l,k-1}$, and let $\mathbf{w}_{n,l,k} = \mathbf{w}_{n,l,k}^{(\tau)}$. Let $b = |\zeta_{n,l,k}^{(t)}|$ be the batch size. After local training, the server selects a subset of clients $\mathcal{S}_{l,k} \subseteq \mathcal{N}$ to participate in the global model aggregation, and let $S_{l,k} = |\mathcal{S}_{l,k}|$. The server aggregates them to obtain a new global model $\mathbf{w}_{l,k} = \sum_{n \in \mathcal{S}_{l,k}} \alpha_{n,l,k} \mathbf{w}_{n,l,k}$, where $\alpha_{n,l,k} = \frac{D_{n,l}}{\sum_{n \in \mathcal{S}_{l,k}} D_{n,l}}$.

B. Delay Model

1) *Local training delay*: According to [35]–[37], the local training delay $t_{n,l,k}^{\text{cp}}$ is characterized by a shifted exponential distribution, capturing both deterministic and stochastic variations arising from client heterogeneity, system load fluctuations, background processes, and thermal effects. The cumulative distribution function (CDF) of the computation delay is given by:

$$\mathbb{P}[t_{n,l,k}^{\text{cp}} < t] = \begin{cases} 1 - e^{-\frac{\mu_n}{\tau b} (t - a_n \tau b)}, & t \geq a_n \tau b, \\ 0, & \text{otherwise,} \end{cases} \quad (4)$$

where $a_n > 0$ represents the minimum computation time per sample for client n and μ_n characterizes the random computation delay parameter for client n .

2) *Model uploading delay*: The communication phase is modeled using a wireless frequency division multiple access system, where clients upload local model updates to the server and the total bandwidth B is dynamically allocated to optimize communication efficiency.

The transmission rate of client n at round k follows the Shannon formula $R_{n,l,k} = B_{n,l,k} \log_2 \left(1 + \frac{P_n |H_{n,l,k}|^2}{B_{n,l,k} N_0} \right)$, where $B_{n,l,k}$ represents the bandwidth allocated to client n at round k of frame l as an optimization variable, satisfying $\sum_{n \in \mathcal{S}_{l,k}} B_{n,l,k} \leq B$. P_n denotes the transmission power of client n which is typically constrained by battery capacity, $|H_{n,l,k}|^2$ captures the average channel gain during the transmission period that incorporates path loss, shadowing, and fading effects, and N_0 represents the noise power spectral density of the wireless channel. The communication delay for uploading the local model vector is $t_{n,l,k}^{\text{cm}} = \frac{D_s}{R_{n,l,k}}$, where D_s represents the total data size of model vector in bits.

3) *Total training delay and its constraint*: We consider synchronous model aggregation in this work, and thus each round is constrained by the slowest scheduled client. The total training delay of round k is determined by the maximum computation and communication time among all participating clients:

$$t_{l,k}(\mathcal{S}_{l,k}, \mathbf{B}_{l,k}) = \max_{n \in \mathcal{S}_{l,k}} \{t_{n,l,k}^{\text{cp}} + t_{n,l,k}^{\text{cm}}\}, \quad (5)$$

where $\mathcal{S}_{l,k} \subseteq \mathcal{N}$ denotes the set of scheduled clients in round k of frame l , and $\mathbf{B}_{l,k} = [B_{1,l,k}, B_{2,l,k}, \dots, B_{N,l,k}]$ represents the bandwidth allocation vector for round k in frame l .

To guarantee timely learning, we impose a delay constraint $T_{\max}$ in each round, so that stale clients cannot be scheduled:

$$t_{l,k}(\mathcal{S}_{l,k}, \mathbf{B}_{l,k}) \leq T_{\max}. \quad (6)$$

C. Problem Formulation

Define $\tilde{\mathbf{w}}_l \triangleq \arg \min_{\mathbf{w} \in \{\mathbf{w}_{l,k}, k=0,1,2,\dots,K_l\}} F_l(\mathbf{w}, \mathcal{D}_l)$, which is the optimal model parameter that achieves the minimum global loss function value throughout the training process in frame l .

The goal is to achieve timely FEEL when new data arrives in each frame. Specifically, given the training delay budget

$T_{\max}$ and the previous frame model $\tilde{\mathbf{w}}_{l-1}$, we aim to minimize the expected global loss function $\mathbb{E}[F_l(\tilde{\mathbf{w}}_l, \mathcal{D}_l) | \tilde{\mathbf{w}}_{l-1}]$, by optimizing the client scheduling strategy $\mathcal{S}_{l,k}$ and spectrum allocation strategy $\mathbf{B}_{l,k}$. The problem is formulated as

$$\mathbf{P1}: \min_{\mathcal{S}_{l,k}, \mathbf{B}_{l,k}} \mathbb{E}[F_l(\tilde{\mathbf{w}}_l, \mathcal{D}_l) | \tilde{\mathbf{w}}_{l-1}] \quad (7a)$$

$$\text{s.t. } t_{l,k}(\mathcal{S}_{l,k}, \mathbf{B}_{l,k}) \leq T_{\max}, \quad \forall l, \quad (7b)$$

$$\sum_{n \in \mathcal{S}_{l,k}} B_{n,l,k} \leq B, \quad \forall l, k, \quad (7c)$$

$$\mathcal{S}_{l,k} \subseteq \mathcal{N}, B_{n,l,k} \geq 0, \forall n, l, k. \quad (7d)$$

There are two main challenges to solve **P1**. First, the objective function is not tractable, and is affected by various factors such as data distributions, scheduling, and bandwidth allocation. Second, since the update on the new dataset depends on the previously trained model $\tilde{\mathbf{w}}_{l-1}$, the old and new data may exert different influences on the training process, which remains unclear in the current literature on FEEL.

IV. METHODOLOGY

In this section, we solve **P1** by proposing a temporal-drift-and-divergence-aware client scheduling algorithm called FedTeddi. To address the challenges posed by temporal data evolution and client heterogeneity in FEEL, we first formally define the concepts of temporal drift and collective divergence through gradient-based characterization methods and derive closed-form expressions for classification tasks. Then, we propose an optimization objective which strikes a balance between temporal drift and collective divergence for fast convergence. Finally, we develop a greedy scheduling and bandwidth allocation algorithm that efficiently selects clients while satisfying delay constraints, forming the complete FedTeddi framework.

A. Temporal Drift to Quantify Data Evolution

In FEEL, data streams are continuous and dynamic, with each client continuously receiving new data samples. When new data arrives, model training encounters two critical challenges: over-reliance on historical data may limit effective acquisition of new knowledge, preventing the model from timely adaptation to changing data distributions; conversely, excessive emphasis on new data may lead to catastrophic forgetting of valuable historical knowledge. Moreover, data evolution in real-world environments is often non-uniform, with significant variations in the degree of data distribution changes across clients, further exacerbating learning complexity. Thus, balancing *plasticity* (learning new knowledge) and *stability* (retaining old knowledge) is a key challenge in scenarios with continuously evolving data.

To quantify the impact of local data evolution over time on model learning, we introduce the definition of temporal drift. **Definition 1 (Temporal Drift)** For any client n and frame l , there exists a temporal drift bound $\xi_{n,l}$ such that for any model vector $\mathbf{w}$:

$$\|\nabla F_{n,l}(\mathbf{w}, \mathcal{D}_{n,l}) - \nabla F_{n,l-1}(\mathbf{w}, \mathcal{D}_{n,l-1})\| \leq \xi_{n,l}. \quad (8)$$

Here, $\xi_{n,l}$ characterizes the upper bound on the maximum difference in local gradient changes at client n between frames $l-1$ and l . $\|\cdot\|$ denotes the L2-norm of a vector.

Remark 1. Temporal drift essentially reflects the variation in data distribution across consecutive frames for clients. We employ gradient variations to characterize the impact of temporal drift. Gradients directly determine the direction and magnitude of model parameter updates. When data distribution changes, gradient variations can sensitively capture the impact of such changes on model performance. If the gradient difference between consecutive frames for the same client is small, it indicates that the new and old data distributions are relatively similar, with limited impact on model performance. Conversely, if the gradient difference is large, it implies that the model's fitting performance on new data under current parameters is poor, and the new data contains substantial information not yet mastered by the model. The model needs to actively absorb these data samples to promote knowledge acquisition. Therefore, this temporal drift can effectively characterize the learning value of new data for the model.

For classification tasks, temporal drift can be further characterized through changes in class distributions. The difference in class distributions between consecutive time periods can be measured using EMD, leading to the following lemma:

Lemma 1. For classification tasks, the temporal drift can be further bounded by:

$$\begin{aligned} & \|\nabla F_{n,l}(\mathbf{w}, \mathcal{D}_{n,l}) - \nabla F_{n,l-1}(\mathbf{w}, \mathcal{D}_{n,l-1})\| \\ & \leq \sum_{c=1}^{C_l} \|p_{n,l}^{(c)} - p_{n,l-1}^{(c)}\| L^{(c)}, \end{aligned} \quad (9)$$

where $L^{(c)} = \|\nabla \mathbb{E}_{\mathbf{x}|y=c} [f(\mathbf{w}; \mathbf{x}, y)]\|$ denotes the gradient norm for class c . For any given model $\mathbf{w}$, the expected gradient on class c is assumed to be identical. This indicates a realistic scenario where the data features of each class do not change over time.

Proof: By expanding the expectation over classes,

$$\begin{aligned} & \|\nabla F_{n,l}(\mathbf{w}_l, \mathcal{D}_{n,l}) - \nabla F_{n,l-1}(\mathbf{w}_l, \mathcal{D}_{n,l-1})\| \\ & = \left\| \sum_{c=1}^{C_l} p_{n,l}^{(c)} \nabla \mathbb{E}_{\mathbf{x}|y=c} [f(\mathbf{w}_l; \mathbf{x}, y)] \right. \\ & \quad \left. - \sum_{c=1}^{C_{l-1}} p_{n,l-1}^{(c)} \nabla \mathbb{E}_{\mathbf{x}|y=c} [f(\mathbf{w}_l; \mathbf{x}, y)] \right\| \\ & = \left\| \sum_{c=1}^{C_l} (p_{n,l}^{(c)} - p_{n,l-1}^{(c)}) \nabla \mathbb{E}_{\mathbf{x}|y=c} [f(\mathbf{w}_l; \mathbf{x}, y)] \right\| \\ & \leq \sum_{c=1}^{C_l} \|p_{n,l}^{(c)} - p_{n,l-1}^{(c)}\| \|\nabla \mathbb{E}_{\mathbf{x}|y=c} [f(\mathbf{w}_l; \mathbf{x}, y)]\| \\ & = \sum_{c=1}^{C_l} \|p_{n,l}^{(c)} - p_{n,l-1}^{(c)}\| L^{(c)}, \end{aligned}$$

where the inequality holds because of triangle inequality. If a new class c_{new} appears in frame l but was absent in frame $l-1$, we define its prior proportion $p_{n,l-1}^{(c_{\text{new}})} = 0$.

Lemma 1 indicates that temporal drift is determined by the weighted sum of distribution changes across all classes, with weights being the gradient norms of respective classes. The bound is in fact the EMD between the class distributions of current frame l and the last frame $l-1$.

B. Collective Divergence to Quantify Non-i.i.d. Impact

In each round of federated learning, it is desired that the local data across the *scheduled clients collectively form a sample set with global representativeness*. This helps ensure that local update directions align with the global objective and that the model is trained effectively across all classes. However, under data heterogeneity, scheduling based solely on individual client divergence may still lead to a joint data distribution that significantly deviates from the global one, even if some clients exhibit small individual differences. As a result, the aggregated model update may diverge from the optimal global direction. On the other hand, a set of clients that are *complementary* in data or gradient space may produce an averaged update direction closer to the global gradient, even when their individual divergences are relatively large.

Therefore, to properly evaluate the aggregation bias introduced by the set of scheduled clients as a whole, it is necessary to consider the overall distributional characteristics of the selected group. Toward this end, we introduce the concept of collective divergence.

Definition 2 (Collective Divergence) In round k of frame l , given the set of scheduled clients $\mathcal{S}_{l,k}$, the collective divergence is defined as

$$\delta_{l,k} = \left\| \sum_{n \in \mathcal{S}_{l,k}} \alpha_{n,l,k} \nabla F_{n,l}(\mathbf{w}_{l,k}, \mathcal{D}_{n,l}) - \nabla F_l(\mathbf{w}_{l,k}, \mathcal{D}_l) \right\|. \quad (10)$$

$\delta_{l,k}$ represents the discrepancy between the weighted average of gradients from the scheduled clients and the global gradient. A small collective divergence indicates that the joint data distribution of the selected clients possesses good global representativeness, and that their average gradient direction closely approximates the true gradient over the global dataset.

Similar to Lemma 1, collective divergence $\delta_{l,k}$ in classification problems can also be characterized by the EMD between class distributions:

$$\delta_{l,k} \leq \sum_{c=1}^{C_l} \left\| \sum_{n \in \mathcal{S}_{l,k}} \alpha_{n,l,k} p_{n,l-1,k}^{(c)} - p_{l-1,k}^{(c)} \right\| L^{(c)}. \quad (11)$$

By selecting clients with complementary class distributions that are closer to the global distribution after aggregation, we can effectively reduce collective divergence $\delta_{l,k}$, thereby improving model convergence performance. Following our previous work [30], the impact of $\delta_{l,k}$ is derived in the following theorem.

Theorem 1 (Convergence Upper Bound) Assume that loss function $F_{n,l}(\mathbf{w})$ is convex, ρ -Lipschitz, and β -smooth, with uniformly bounded gradient variance $\mathbb{E} \|\nabla F_{n,l}(\mathbf{w})\|^2 \leq g^2$ and unbiased stochastic gradients satisfying $\mathbb{E}[\|\nabla F_{n,l}(\mathbf{w}, \mathbf{x}) -$

$\nabla F_{n,l}(\mathbf{w})\|^2] \leq \sigma^2$. Under learning rate $\eta \leq \frac{1}{\beta}$, given any client scheduling patterns $\{\mathcal{S}_{l,k}, k = 1, \dots, K_l\}$, the loss function after K_l rounds is bounded by

$$F_l(\mathbf{w}_{l,K_l}) - F_l(\mathbf{w}_l^*) \leq \frac{1}{K_l \omega \eta \tau \left(1 - \frac{\beta \eta}{2}\right) - \sum_{k=1}^{K_l} \frac{\rho \mathbb{E}[V_{l,k}]}{\epsilon^2}},$$

where $\mathbf{w}_l^*$ is the optimal model vector, ω and ϵ are positive parameters. $V_{l,k} = \|\mathbf{w}_{l,k} - \mathbf{v}_{l,k}\|$ represents the difference in the convergence speed between federated and centralized learning with $\mathbf{v}_{l,k} = \mathbf{w}_{l,k-1} - \eta \tau \nabla F_l(\mathbf{w}_{l,k-1})$. The $V_{l,k}$ is upper bounded by:

$$\begin{aligned} \mathbb{E}[V_{l,k}] &\leq \underbrace{\frac{1}{2} \tau (\tau - 1) \eta \beta g}_{\text{Iteration Error}} + \underbrace{\eta \tau \frac{\sigma}{\sqrt{S_{l,k} b}}}_{\text{Sampling Variance}} \\ &\quad + \underbrace{\eta \tau \sum_{c=1}^{C_l} \left\| \sum_{n \in \mathcal{S}_{l,k}} \alpha_{n,l,k} p_{n,l,k}^{(c)} - p_{l,k}^{(c)} \right\| L^{(c)}}_{\text{Collective Divergence}}. \end{aligned} \quad (12)$$

Theorem 1 can be obtained following similar derivations as our previous work [30]. We omit the proof due to space limitations while making the following remark.

Remark 2. The convergence upper bound indicates that the convergence speed is predominantly governed by the collective divergence and the sampling variance when the local iteration τ and the learning rate η are fixed. The smaller the collective divergence $\delta_{l,k}$, the greater the performance gap between federated and centralized. Samples within each client are selected through random sampling, and the sampling variance term is inversely proportional to the square root of the number of scheduled clients. If more clients are scheduled to reduce sampling variance, a larger collective divergence may be introduced due to the selection of more heterogeneous clients. Therefore, a joint optimization between sampling variance and collective divergence is necessary to improve overall convergence performance.

C. Problem Transformation

Based on the above analysis, temporal drift focuses on temporal evolution within individual clients, while collective divergence concerns distribution differences among multiple clients. In fact, temporal drift and collective divergence are intrinsically related. When clients have identical initial class distributions and all clients introduce the same number of new classes, no data heterogeneity exists among clients, making scheduling meaningless. However, when clients are inherently data-heterogeneous, temporal drift also affects the degree of heterogeneity. If the temporal drift among clients is convergent (e.g., all moving closer to the global distribution), heterogeneity is reduced, diminishing the collective divergence. If their drifts are divergent (e.g., each skewing towards different classes), heterogeneity is exacerbated, increasing collective divergence, which necessitates scheduling complementary clients to counteract this effect. Therefore, we need to balance these two factors.

We reformulate the original problem **P1** as **P2**, with the objective of directly minimizing sampling variance and collective divergence, while introducing a temporal drift term as an *exploration incentive* to encourage the model to select clients with significant data distribution changes. Specifically,

$$\begin{aligned} \mathbf{P2}: \min_{S_{l,k}, B_{l,k}} & \frac{\sigma}{\sqrt{S_{l,k}b}} + \sum_{c=1}^{C_l} L^{(c)} \left\| \sum_{n \in S_{l,k}} \alpha_{n,l,k} p_{n,l,k}^{(c)} \right. \\ & \left. - p_{l,k}^{(c)} \right\| - \lambda_k \sum_{c=1}^{C_l} \sum_{n \in S_{l,k}} L^{(c)} \alpha_{n,l,k} \left\| p_{n,l,k}^{(c)} - p_{n,l-1,k}^{(c)} \right\| \quad (13) \\ \text{s.t.} & \text{ constraints (7b), (7c), (7d),} \end{aligned}$$

where $\lambda_k = \lambda_0 \left(1 - \frac{k}{K_l}\right)$ is a decaying weight that balances the original data loss and the drift penalty over time.

Remark 3. The three terms in the objective function correspond to sampling variance, collective divergence, and temporal drift, respectively. Minimizing the first two terms ensures learning stability, while the temporal drift term with a negative sign encourages selection of clients with significant data distribution changes. However, as training progresses, to prevent catastrophic forgetting, we introduce a time-varying weight λ_k before the temporal drift term to achieve plasticity-stability trade-off: in the early stages of each frame, we select clients with significant data distribution changes to guide the model in learning new knowledge from new data and adapting to data stream changes; as frame training progresses and sufficient new knowledge is acquired, the weight decreases to prevent catastrophic forgetting of historical knowledge.

D. FedTeddi: A Temporal-Drift-and-Divergence-Aware Client Scheduling Algorithm

The primary difficulty in solving **P2** stems from the combinatorial structure of the objective function. That is, the scheduled clients *jointly* determine the collective divergence and temporal drift. Furthermore, bandwidth allocation is coupled with the binary scheduling indicator, which makes the problem more challenging. Therefore, we first address the bandwidth allocation problem under given scheduling decisions, then solve it through greedy scheduling.

1) *Bandwidth allocation*: According to Theorem 1 in [35], the minimum bandwidth for each scheduled client in **P2** is given by

$$B_{n,l,k}^* = - \frac{D_s \ln 2}{(T_{\max} - t_{n,l,k}^{\text{cp}}) (W(-\Gamma_n e^{-\Gamma_n}) + \Gamma_n)}, \quad (14)$$

where D_s denotes the data size of the model in bits, $\Gamma_n = \frac{N_0 D_s \ln 2}{(T_{\max} - t_{n,l,k}^{\text{cp}}) P_n |H_{n,l,k}|^2}$ and $W(\cdot)$ is the Lambert-W function.

2) *Greedy scheduling algorithm*: Combining the temporal drift and collective divergence into a unified function $\mathcal{U}(S_{l,k})$,

we define

$$\begin{aligned} \mathcal{U}(S_{l,k}) = & \sum_{c=1}^{C_l} \left\| \sum_{n \in S_{l,k}} \alpha_{n,l,k} p_{n,l,k}^{(c)} - p_{l,k}^{(c)} \right\| L^{(c)} \\ & - \lambda_k \sum_{c=1}^{C_l} \sum_{n \in S_{l,k}} \alpha_{n,l,k} \left\| p_{n,l,k}^{(c)} - p_{n,l-1,k}^{(c)} \right\| L^{(c)}. \quad (15) \end{aligned}$$

Substituting (14) into **P2**, the client scheduling problem is transformed as:

$$\begin{aligned} \mathbf{P3}: \min_{S_{l,k}} & \frac{\sigma}{\sqrt{S_{l,k}b}} + \mathcal{U}(S_{l,k}) \quad (16) \\ \text{s.t.} & \sum_{n \in S_{l,k}} B_{n,l,k}^* \leq B, \quad \forall l, k, \\ & S_{l,k} \subseteq \mathcal{N}, B_{n,l,k}^* \geq 0, \forall n, l, k. \end{aligned}$$

P3 is still a combinatorial optimization problem, which is difficult to solve. Next, we propose a greedy scheduling method to approximately solve the problem.

The greedy scheduling strategy operates through iterative client selection based on objective function improvement. We first initialize the scheduling set $S_{l,k} = \emptyset$ and the candidate client pool $\mathcal{M} = \mathcal{N}$, then proceed through repeated selection iterations.

In each iteration, we evaluate the gain of adding each candidate client to the current scheduling set by computing $\mathcal{U}(S_{l,k} \cup n) - \mathcal{U}(S_{l,k})$. We select the client with the minimum value $n^* = \arg \min_{n \in \mathcal{M}} [\mathcal{U}(S_{l,k} \cup n) - \mathcal{U}(S_{l,k})]$.

We then compute the overall objective function change after adding client n^* . The complete condition for client selection includes both the unified objective function change and the sampling variance term:

$$\mathcal{U}(S_{l,k} \cup n^*) - \mathcal{U}(S_{l,k}) + \frac{\sigma}{\sqrt{b}} \left(\frac{1}{\sqrt{S_{l,k}+1}} - \frac{1}{\sqrt{S_{l,k}}} \right) \leq 0. \quad (17)$$

If this condition is satisfied, we calculate the optimal bandwidth $B_{n^*,l,k}^*$ according to (14). Subsequently, we check the bandwidth constraint $\sum_{m \in S_{l,k}} B_{m,l,k}^* + B_{n^*,l,k}^* \leq B$. If the addition of this client can reduce the objective function and the sum of currently allocated bandwidth plus $B_{n^*,l,k}^*$ does not exceed the total bandwidth B , we add n^* to the scheduling set and remove it from the candidate pool $S_{l,k} \leftarrow S_{l,k} \cup n^*$, $\mathcal{M} \leftarrow \mathcal{M} \setminus n^*$.

This process is repeated until no client can bring benefits or resources are insufficient. Finally, we obtain the scheduled client set $S_{l,k}$.

3) *Parameter estimation*: Since we cannot collect all data in a single round to compute the global loss and gradient, the precise values of gradient variance σ and gradient norm $L^{(c)}$ of class c cannot be obtained. We can estimate σ by computing a weighted average of the gradient variance $\hat{\sigma}_{n,l,k}$ using the first batch of data from each client.

Meanwhile, $L^{(c)}$ in round k of frame l can be estimated as

$$\hat{L}_{l,k}^{(c)} = \max_{n \in S_{l,k}} \frac{\left\| \nabla F_{n,l}(\mathbf{w}_{l,k}, \mathcal{D}_{n,l}^{(c)}) - \nabla F_l(\mathbf{w}_{l,k}, \mathcal{D}_l^{(c)}) \right\|}{\left\| \mathbf{p}_{n,l,k} - \mathbf{p}_{l,k} \right\|_1}, \quad (18)$$

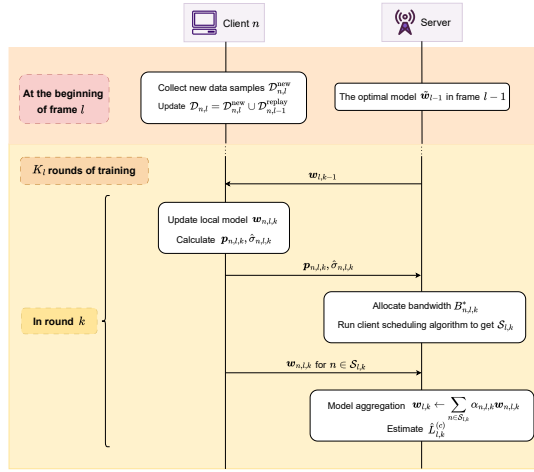


Fig. 2. The workflow of the FedTeddi within a single frame l .

where $\mathcal{D}_{n,l}^{(c)}$ and $\mathcal{D}_l^{(c)}$ denote the local dataset and global dataset that contain only samples from class c . $\hat{L}_{l,k}^{(c)}$ is derived as follows.

Assume that client n contains only class c of data. The local gradient of client n is $\nabla F_{n,l}(\mathbf{w}_{l,k}, \mathcal{D}_{n,l}^{(c)}) = p_{n,l,k}^{(c)} \nabla \mathbb{E}_{\mathbf{x}|y=c} [f(\mathbf{w}_{l,k}; \mathbf{x}, c)]$. The global gradient of class c is $\nabla F_l(\mathbf{w}_{l,k}, \mathcal{D}_l^{(c)}) = p_{l,k}^{(c)} \nabla \mathbb{E}_{\mathbf{x}|y=c} [f(\mathbf{w}_{l,k}; \mathbf{x}, c)]$.

Let $L_{l,k}^{(c)} = \|\nabla \mathbb{E}_{\mathbf{x}|y=c} [f(\mathbf{w}_{l,k}; \mathbf{x}, y)]\|$. On a given class c , the global-local gradient difference is:

$$\begin{aligned} & \left\| \nabla F_{n,l}(\mathbf{w}_{l,k}, \mathcal{D}_{n,l}^{(c)}) - \nabla F_l(\mathbf{w}_{l,k}, \mathcal{D}_l^{(c)}) \right\| \\ &= \left\| p_{n,l,k}^{(c)} \nabla \mathbb{E}_{\mathbf{x}|y=c} [f(\mathbf{w}_{l,k}; \mathbf{x}, c)] - p_{l,k}^{(c)} \nabla \mathbb{E}_{\mathbf{x}|y=c} [f(\mathbf{w}_{l,k}; \mathbf{x}, c)] \right\| \\ &= \left\| p_{n,l,k}^{(c)} - p_{l,k}^{(c)} \right\|_1 L_{l,k}^{(c)}. \end{aligned}$$

Therefore $L_{l,k}^{(c)} = \frac{\left\| \nabla F_{n,l}(\mathbf{w}_{l,k}, \mathcal{D}_{n,l}^{(c)}) - \nabla F_l(\mathbf{w}_{l,k}, \mathcal{D}_l^{(c)}) \right\|}{\left\| p_{n,l,k}^{(c)} - p_{l,k}^{(c)} \right\|_1}$. To ensure

that $\hat{L}_{l,k}^{(c)} \geq L_{l,k}^{(c)}$, which preserves the validity of our convergence upper bound, we select the maximum value across all scheduled clients. When clients contain multiple classes, deriving $L_{l,k}^{(c)}$ becomes difficult, so we adopt the same estimation approach as the single-class case with the denominator represented by the class distribution vector.

4) *FedTeddi algorithm*: Based on the above steps, we now present our FedTeddi algorithm that integrates temporal drift and collective divergence for client scheduling and bandwidth allocation, as shown in Algorithm 1.

At the beginning of each frame, clients collect new data and update their local datasets. Within each frame, multiple communication rounds are executed. The server first broadcasts the global model to all clients, then each client performs τ steps of local training and sends the gradient variance $\hat{\sigma}_{n,l,k}$ and all batches distribution $p_{n,l,k}$ to the server (lines 5-11). We employ a greedy client selection mechanism and the server then obtains the optimal client subset according to the scheduling strategy (lines 12-24). Then, the selected clients

Algorithm 1 The FedTeddi Algorithm

- 1: **Input:** The optimal model $\tilde{w}_{l-1}$ obtained in frame $l-1$, local dataset $\mathcal{D}_{n,l} = \mathcal{D}_{n,l}^{\text{new}} \cup \mathcal{D}_{n,l-1}^{\text{replay}}$. Initialize $w_{l,0} = \tilde{w}_{l-1}$.
- 2: **Output:** Final global $\tilde{w}_l$ of frame l .
- 3: **for** round $k = 1, 2, \dots, K_l$ **do**
- 4: Initialize $S_{l,k} = \emptyset, \mathcal{M} = \mathcal{N}$.
- 5: Server broadcasts model $w_{l,k-1}$ to all clients $\mathcal{N}$.
- 6: **for** each client $n \in \mathcal{N}$ **do**
- 7: Local training for τ iterations according to (3).
- 8: Estimate gradient variance $\hat{\sigma}_{n,l,k}$.
- 9: Calculate class distribution $p_{n,l,k}$ of all batches.
- 10: Send $p_{n,l,k}, \hat{\sigma}_{n,l,k}$ to the server.
- 11: **end for**
- 12: Server receives all $p_{n,l,k}, \hat{\sigma}_{n,l,k}$ from all clients, and computes $\hat{\sigma}_{l,k} \leftarrow \sum_{n \in \mathcal{N}} \alpha_{n,l,k} \hat{\sigma}_{n,l,k}$.
- 13: **while** $|\mathcal{M}| > 0$ **do**
- 14: Find $n^* = \arg \min_{n \in \mathcal{M}} [\mathcal{U}(S_{l,k} \cup n) - \mathcal{U}(S_{l,k})]$.
- 15: **if** (17) is satisfied **then**
- 16: Calculate optimal bandwidth $B_{n^*,l,k}^*$ by (14).
- 17: **if** $\sum_{m \in S_{l,k}} B_{m,l,k}^* + B_{n^*,l,k}^* \leq B$ **then**
- 18: Update $S_{l,k} \leftarrow S_{l,k} \cup n^*, \mathcal{M} \leftarrow \mathcal{M} \setminus n^*$.
- 19: **end if**
- 20: **else**
- 21: break
- 22: **end if**
- 23: **end while**
- 24: Obtain the scheduled client set $S_{l,k}$.
- 25: **for** each client $n \in S_{l,k}$ **do**
- 26: Send local model $w_{n,l,k}$ to the server.
- 27: **end for**
- 28: Server aggregates $w_{l,k} \leftarrow \sum_{n \in S_{l,k}} \alpha_{n,l,k} w_{n,l,k}$.
- 29: Server estimates $\hat{L}_{l,k}^{(c)}$ according to (18).
- 30: **end for**

upload their local models and the server performs model aggregation using weighted averaging over the selected clients (lines 25-28). Finally, the server estimates $\hat{L}_{l,k}^{(c)}$ according to (18) (line 29). To better illustrate this process, Fig. 2 shows the workflow of FedTeddi within a single frame.

V. EXPERIMENTS

A. Experimental Setup

We consider a FEEL system comprised of a single edge server and $N = 30$ clients. These clients are randomly distributed within a circular area of radius 250m centered at the server. The total system bandwidth is $B = 20\text{MHz}$. The transmit power of each client is fixed at 23dBm. The noise power spectral density is -174dBm/Hz , and the carrier frequency is 3.5GHz.

The wireless channel model incorporates both path loss and shadowing. The path loss is given by $128.1 + 37.6 \log_{10}(d)$, where d is the distance between the client and the server in kilometers. Shadow fading follows a log-normal distribution

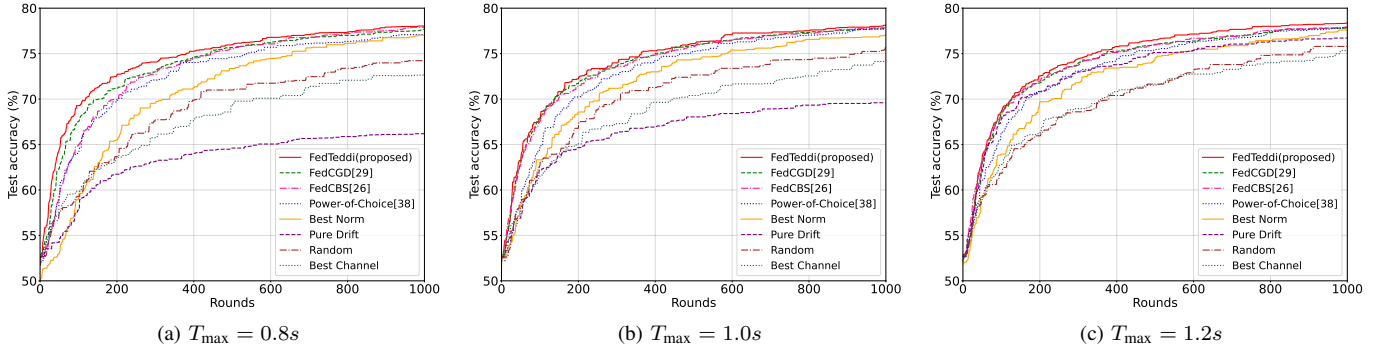


Fig. 3. Test accuracy of baselines with different delay constraints.

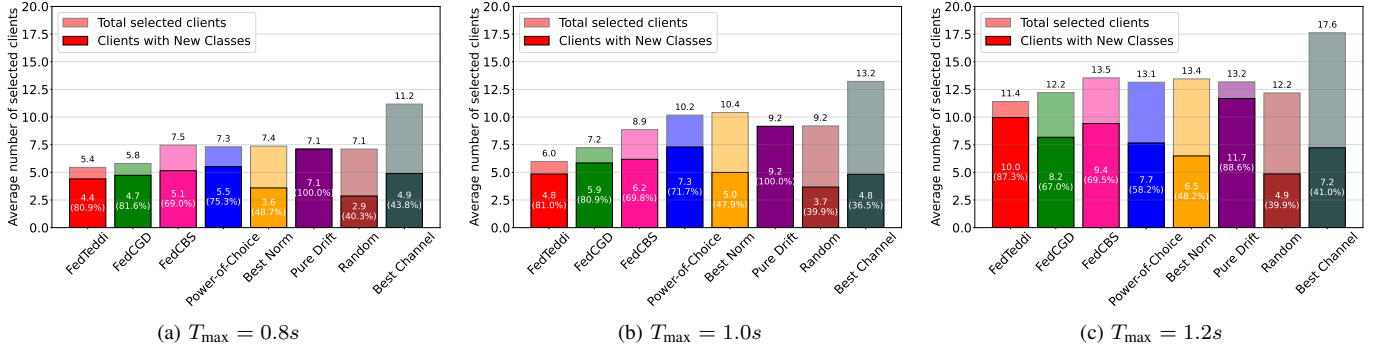


Fig. 4. Client selection results with different delay constraints.

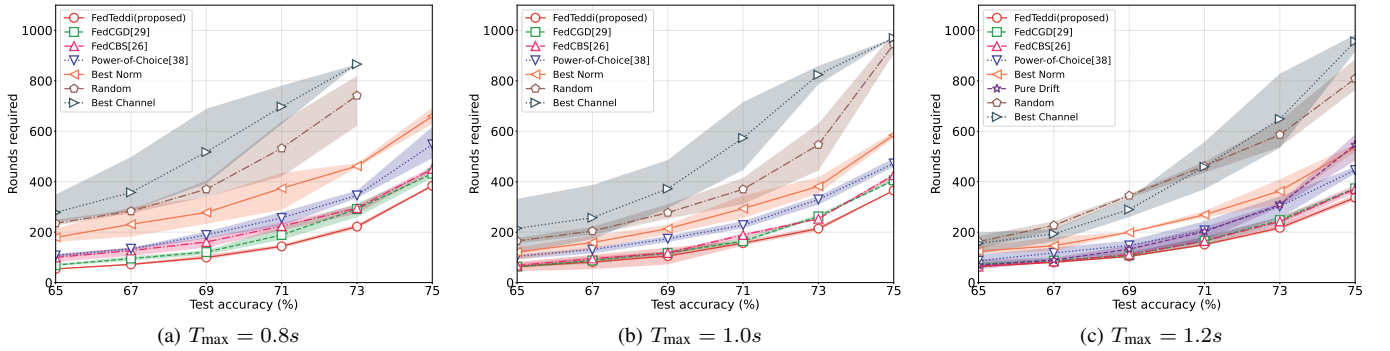


Fig. 5. Rounds to reach different accuracy levels with different delay constraints. The results are averaged over 5 independent runs, and the shadow areas represent the standard deviation.

with a standard deviation of 8dB. The computation delay of client n is modeled by a shifted exponential distribution with parameters $a_i = 0.5\text{ms/sample}$ and $\mu_i = 2.0\text{samples/ms}$.

Simulations are conducted on two image classification datasets: CIFAR-10 [40] and CIFAR-100 [40]. For the CIFAR-10 dataset, each client always stores 750 samples, while for CIFAR-100, each client has 300 samples. When new data arrives, each client randomly selects a subset of samples from its existing dataset based on the storage limit. The training delay deadline $T_{\max}$ is set to 0.8s, 1.0s, and 1.2s, respectively. For the CIFAR-100 dataset, $T_{\max}$ is set to 30s.

CIFAR-10 Setting: An initial model is pre-trained on classes 0-5. In frame 0, the global dataset is imbalanced. The total number of samples from classes 0-2 is three times that of classes 3-5. 20 clients each has one class of data and 10

clients each has two classes of data with each class of data has the same samples. At the beginning of frame 1, 12 randomly selected clients each collect a new class of data with a sample size of 375, while randomly discarding a portion of old data. The datasets of the other 18 clients remain unchanged.

CIFAR-100 Setting: The initial model is pre-trained on classes 0-29. In frame 0, the global dataset is imbalanced. 10 clients each has one class of data and 20 clients each has two classes of data with different samples. The learning process unfolds in two frames. In frame 1, 12 clients are randomly selected to introduce classes 30-39, each client has one new class of data with a sample size of 150, while randomly discarding a portion of old data. The other 18 clients remain unchanged. In frame 2, also 12 clients are randomly selected to introduce classes 40-49 in the same manner as in frame 1.

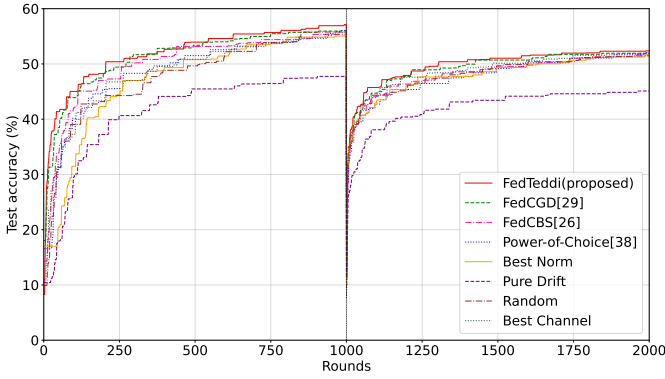


Fig. 6. Test accuracy of baselines on CIFAR-100.

For CIFAR-10, we adopt a convolutional neural network (CNN) architecture consisting of two convolutional blocks and fully connected layers. The first convolutional block contains two 3×3 convolutional layers (32 channels), ReLU activation, 2×2 max pooling, and 0.2 dropout; the second convolutional block contains two 3×3 convolutional layers (64 channels), ReLU activation, 2×2 max pooling, and 0.3 dropout; the feature extractor is a 120-unit fully connected layer with ReLU activation. For CIFAR-100, we adopt the ResNet-18 architecture. For both networks, all Batch Normalization layers are replaced with Group Normalization layers. We use SGD with a momentum of 0.5. The initial learning rate is set to 0.01 and decays exponentially after each round with a factor of 0.9992. The local batch size is set to 32 for all clients.

The following benchmarks are compared:

- (1) **Random**: Randomly select clients uniformly at random under bandwidth constraints.
- (2) **Best Channel [38]**: Prioritize clients with the highest channel gain.
- (3) **Best Norm [38]**: Select clients with the largest L2-norm of local gradient updates.
- (4) **Power-of-Choice [39]**: Randomly select a subset of clients and prioritize them with higher local training loss.
- (5) **Pure Drift**: Rank and select clients purely based on the magnitude of their data distribution drift.
- (6) **FedCBS [27]**: Employ a probability sampling based on a novel quadratic class-imbalance degree (QCID) metric.
- (7) **FedCGD [30]**: Optimize client selection by minimizing the multi-level collective divergences measured through the weighted EMD.

In particular, Pure Drift and FedCGD serve as ablation variants of our method.

B. Results on CIFAR-10

The performance of different client selection strategies on CIFAR-10 under varying training delay deadlines is illustrated in Fig. 3–Fig. 5. Overall, our proposed FedTeddi consistently outperforms all baselines in terms of test accuracy, convergence speed and robustness.

As shown in Fig. 3, FedTeddi achieves the fastest convergence and the highest final test accuracy across all three

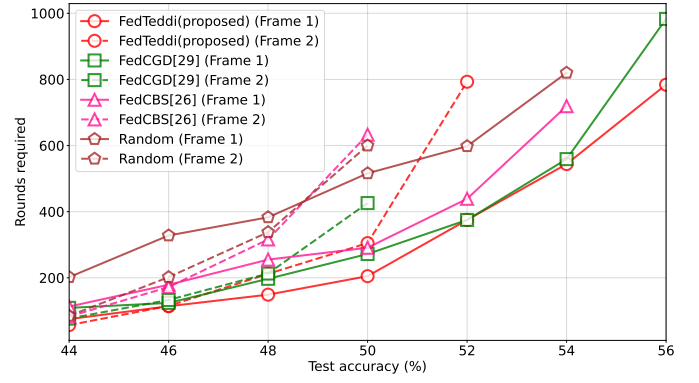


Fig. 7. Rounds to reach different accuracy levels on CIFAR-100.

training delay deadlines ($T_{\max} = 0.8, 1.0, 1.2s$). The advantage is particularly significant when delay constraint is tight.

Fig. 4 shows the average number of scheduled clients in frame 1. FedTeddi achieves a better balance between selecting clients with new classes and ensuring overall participation. Compared with random or best channel strategies, FedTeddi schedules more clients containing new classes of data to ensure sufficient learning of new data; while compared with the pure drift strategy, it retains a portion of clients with only old classes of data, thereby avoiding excessive bias toward new knowledge and effectively mitigating the forgetting of old knowledge.

We set $\lambda_0 = 2.0$ and adopt a decaying scheme $\lambda_k = \lambda_0(1 - \frac{k}{K_I})$, which assigns greater weight to the temporal drift term in the early rounds and drives the server to prioritize scheduling clients with new classes of data. At the beginning of each frame, λ_k is reset to λ_0 for adaptation. As shown in Fig. 5, scheduling more clients with new classes in the early rounds significantly accelerates convergence. For example, under $T_{\max} = 1.2s$, the target accuracy of 75% is reached 11.1% faster compared with the FedCGD scheduling and 58.4% over random scheduling. This demonstrates that prioritizing clients with new data helps the model quickly adapt to temporal drift while maintaining high overall accuracy.

C. Results on CIFAR-100

The results on CIFAR-100 are presented in Fig. 6–Fig. 7. FedTeddi consistently demonstrates faster convergence and higher accuracy throughout both frames of the training process. In the early stage of each frame, FedTeddi adapts more quickly to newly introduced classes, while in the later stages, it avoids the drastic performance drops observed in methods such as random and pure drift, which suffer from unstable client selection and excessive drift sensitivity. As shown in Fig. 7, in frame 2, to reach an accuracy of 50%, FedTeddi outperforms random scheduling by using 49.2% fewer rounds, FedCGD by 28.4%, and FedCBS by 51.8%. These results highlight that FedTeddi achieves more stable training in the FEEL system with newly arriving data and data heterogeneity.

VI. CONCLUSION

In this paper, we have proposed FedTeddi for timely model updates of FEEL under dynamic data evolution and

communication resource limits. We have quantified two key factors, namely temporal drift and collective divergence, and characterized their impact on data-evolving FEEL. By integrating the joint awareness of temporal drift and collective divergence, we have developed a joint scheduling and bandwidth allocation algorithm, enabling the FEEL system to learn from new data quickly without forgetting previous knowledge. Experimental results on CIFAR-10 and CIFAR-100 datasets have demonstrated that FedTeddi achieves higher accuracy and accelerates convergence compared with the state-of-the-art benchmarks. Future work will extend FedTeddi to large-scale cross-client deployments and explore its integration with edge-cloud collaborative intelligence.

REFERENCES

- [1] W. Y. B. Lim, N. C. Luong, D. T. Hoang, Y. Jiao, Y. Liang, Q. Yang, D. Niyato, and C. Miao, "Federated learning in mobile edge networks: A comprehensive survey," *IEEE Commun. Surveys Tuts.*, vol. 22, no. 3, pp. 2031–2063, 3rd Quart. 2020.
- [2] Y. Lu et al., "Fed-SCRIP: Federated Multi-View Learning for Seller Claim Risk Prediction in Logistics Scenarios," *2024 IEEE International Conference on High Performance Computing and Communications (HPCC)*, Wuhan, China, 2024, pp. 610–617.
- [3] L. Cao, Y. Zhu, H. Zhou, S. Tan and W. Gong, "Federated Reinforcement Learning for Therapeutic Interventions over ICUs with Noisy Labels," *2025 28th International Conference on Computer Supported Cooperative Work in Design (CSCWD)*, Compiegne, France, 2025, pp. 147–152.
- [4] L. Wang, X. Zhang, H. Su, and J. Zhu, "A comprehensive survey of continual learning: Theory, method and application," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 8, pp. 5362–5383, Aug. 2024.
- [5] Z. Wang, et al., "Federated continual learning for edge-AI: A comprehensive survey," in *arXiv preprint arXiv:2411.13740*, 2024.
- [6] P. Hamed, R. Razavi-Far, and E. Hallaji, "Federated continual learning: Concepts, challenges, and solutions," *arXiv preprint arXiv:2502.07059*, 2025.
- [7] T. Li, A. K. Sahu, A. Talwalkar, and V. Smith, "Federated learning: Challenges, methods, and future directions," *IEEE Signal Process. Mag.*, vol. 37, no. 3, pp. 50–60, May 2020.
- [8] Z. Wang, E. Yang, L. Shen, and H. Huang, "A comprehensive survey of forgetting in deep learning beyond continual learning," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 47, no. 3, pp. 1464–1483, 2025.
- [9] X. Li, K. Huang, W. Yang, S. Wang, and Z. Zhang, "On the convergence of FedAvg on non-i.i.d. data," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, Addis Ababa, Ethiopia, Apr. 2020.
- [10] S. Vahidian, M. Morafah, C. Chen, M. Shah, and B. Lin, "Rethinking data heterogeneity in federated learning: Introducing a new notion and standard benchmarks," *IEEE Trans. Artif. Intell.*, vol. 5, no. 3, pp. 1386–1397, 2024.
- [11] B. Liu, N. Lv, Y. Guo, and Y. Li, "Recent advances on federated learning: A systematic survey," *Neurocomputing*, vol. 597, p. 128019, 2024.
- [12] M. F. Criado, F. E. Casado, R. Iglesias, C. V. Regueiro, and S. Barro, "Non-i.i.d. data and continual learning processes in federated learning: A long road ahead," *Inf. Fusion*, vol. 88, pp. 263–280, 2022.
- [13] C. Chen, T. Liao, X. Deng, Z. Wu, S. Huang, and Z. Zheng, "Advances in robust federated learning: A survey with heterogeneity considerations," *IEEE Trans. Big Data*, pp. 1–20, 2025.
- [14] A. Reisizadeh, I. Tziotis, H. Hassani, A. Mokhtari, and R. Pedarsani, "Straggler-resilient federated learning: Leveraging the interplay between statistical accuracy and system heterogeneity," *IEEE J. Sel. Areas Inf. Theory*, vol. 3, no. 2, pp. 197–205, Jun. 2022.
- [15] Y. Zhou, Y. Shi, H. Zhou, J. Wang, L. Fu, and Y. Yang, "Toward scalable wireless federated learning: Challenges and solutions," *IEEE Internet Things Mag.*, vol. 6, no. 4, pp. 10–16, Dec. 2023.
- [16] Y. Shi, K. Yang, T. Jiang, J. Zhang, and K. B. Letaief, "Communication-efficient edge AI: Algorithms and systems," *IEEE Commun. Surveys Tuts.*, vol. 22, no. 4, pp. 2167–2191, 4th Quart., 2020.
- [17] J. Ren, Y. He, D. Wen, G. Yu, K. Huang, and D. Guo, "Scheduling for cellular federated edge learning with importance and channel awareness," *IEEE Trans. Wireless Commun.*, vol. 20, no. 11, pp. 7690–7703, Nov. 2021.
- [18] Z. Zhang, Y. Li, and J. Liu, "Generative replay with meta-learning for efficient continual adaptation," *Neurocomputing*, vol. 601, pp. 150–162, 2024.
- [19] Y. Li, Q. Li, H. Wang, R. Li, W. Zhong, and G. Zhang, "Towards Efficient Replay in Federated Incremental Learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Seattle, WA, USA, Jun. 2024, pp. 12820–12829.
- [20] C. Sun, L. Wang, and T. Zhang, "Adaptive importance weighting for continual learning," *Pattern Recognit.*, vol. 142, p. 109673, 2023.
- [21] H. Kim, J. Park, and S. Lee, "FedSSI: Rehearsal-Free Continual Federated Learning with Synergistic Synaptic Intelligence," in *Proc. 42nd Int. Conf. Mach. Learn. (ICML)*, Vancouver, Canada, PMLR, 2025.
- [22] Y. Liu, F. Chen, and P. Zhao, "Mask-based continual learning for adaptive neural networks," *Inf. Sci.*, vol. 642, p. 119200, 2023.
- [23] Y. Chen, Z. Chai, Y. Cheng, and H. Rangwala, "FedConD: Asynchronous Federated Learning for Sensor Data with Concept Drift," *arXiv preprint arXiv:2109.00151*, 2021.
- [24] E. Jothimurugesan, K. Hsieh, J. Wang, G. Joshi, and P. B. Gibbons, "Federated Learning under Distributed Concept Drift (FedDrift)," in *Proc. Int. Conf. Artif. Intell. Statist. (AISTATS)*, 2023.
- [25] S. Panchal, P. Chawla, and V. Gupta, "Flash: Fast learning for adaptive concept drift in federated environments," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2023.
- [26] M. Tao, Y. Zhou, Y. Shi, J. Lu, S. Cui, J. Lu, and K. B. Letaief, "Federated edge learning for 6G: Foundations, methodologies, and applications," *Proceedings of the IEEE*, pp. 1–39, 2024.
- [27] L. Zhang, B. He, S. Chen, X.-H. Sun, and Y. Wang, "Fed-CBS: A heterogeneity-aware client sampling mechanism for federated learning via class-imbalance reduction," in *Proc. Int. Conf. Mach. Learn. (ICML)*, Honolulu, HI, USA, Jul. 2023, pp. 41622–41633.
- [28] T. Chen, J. Yan, Y. Sun, S. Zhou, D. Gunduz, and Z. Niu, "Mobility accelerates learning: Convergence analysis on hierarchical federated learning in vehicular networks," *IEEE Trans. Veh. Technol.*, vol. 74, no. 1, pp. 1657–1673, Jan. 2025.
- [29] Y. Sun, S. Zhou, Z. Niu, and D. Gunduz, "Dynamic scheduling for over-the-air federated edge learning with energy constraints," *IEEE J. Sel. Areas Commun.*, vol. 40, no. 1, pp. 227–242, Jan. 2022.
- [30] T. Chen, J. Yan, Y. Sun, S. Zhou, and Z. Niu, "FedCGD: Collective gradient divergence optimized scheduling for wireless federated learning," *arXiv preprint arXiv:2506.07581*, 2025.
- [31] X. Zeng, H. Xu, and M. Chen, "Fairness-Aware Client Selection in Federated Learning with Heterogeneous Data and Resources," *IEEE Trans. Mobile Comput.*, vol. 22, no. 12, pp. 7312–7326, Dec. 2023.
- [32] J. Wang, Y. Liu, and K. Xu, "Reinforcement learning-based client selection for resource-constrained federated learning," *Neurocomputing*, vol. 563, pp. 126–139, 2025.
- [33] J. S. Smith, L. Valkov, S. Halbe, V. Gutta, R. Feris, Z. Kira, and L. Karlinsky, "Adaptive Memory Replay for Continual Learning," in *Proc. 2024 IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Los Alamitos, CA, USA, Jun. 2024, pp. 3605–3615.
- [34] J. Yang, X. Zhou, Y. He, Q. Li, Z. Su, X. Ruan, and C. Zhang, "Effective Generative Replay with Strong Memory for Continual Learning," *Knowledge-Based Systems*, vol. 319, p. 113477, 2025.
- [35] W. Shi, S. Zhou, Z. Niu, M. Jiang, L. Geng, "Joint device scheduling and resource allocation for latency constrained wireless federated learning," *IEEE Trans. Wireless Commun.*, vol. 20, no. 1, pp. 453–467, Sep. 2020.
- [36] S. Li, M. A. Maddah-Ali, and A. S. Avestimehr, "A unified coding framework for distributed computing with straggling servers," in *Proc. IEEE Globecom Workshops (GC Wkshps)*, Dec. 2016, pp. 1–6.
- [37] K. Lee, M. Lam, R. Pedarsani, et al., "Speeding up distributed machine learning using codes," *IEEE Trans. Inf. Theory*, vol. 64, no. 3, pp. 1514–1529, Mar. 2017.
- [38] M. M. Amiri, D. Gündüz, S. R. Kulkarni, and H. V. Poor, "Convergence of update aware client scheduling for federated learning at the wireless edge," *IEEE Trans. Wireless Commun.*, vol. 20, pp. 3643–3658, no. 6, Jun. 2021.
- [39] Y. J. Cho, J. Wang, and G. Joshi, "Towards understanding biased client selection in federated learning," in *Proc. Artificial Intelligence and Statistics (AISTATS)*, May 2022, pp. 10351–10375.
- [40] A. Krizhevsky, "Learning Multiple Layers of Features from Tiny Images," Technical Report, University of Toronto, 2009.